

“SMART VEHICLE LIGHT”

A project report submitted in the partial fulfilment the award degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

By

K.JAHNAVI	-	23U41A0583
K.CHANDINI	-	23U41A0588
P.BHAVANA	-	23U41A0599
G.DHANUSHKA	-	23U41A0578
G.GAGANA SINDHU		23U41A0579
N.AMRUTHA	-	23U41A0598

Under the esteemed Guidance of

DR. K. SUJATHA

HEAD OF DEPARTMENT



**DADI INSTITUTE OF ENGINEERING & TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)**

(Approved by A.I.C.T.E., New Delhi & Permanently Affiliated to JNTU GV)

Accredited by NAAC with ‘A’ Grade and Inclusion u/s 2(f) & 12(B) of UGC Act

An ISO 9001:2015, ISO 14001:2015 & ISO 45001:2018 Certified Institute.

NH-16, Anakapalle – 531002, Visakhapatnam, A.P.

(2021-2025)

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**SMART VEHICLE LIGHT**” submitted by **K.JAHNAVI(23U41A0583), K.CHANDINI(23U41A0588), P.BHAVANA(23U41A0599), G.DHANUSHKA(23U41A0578), G.GAGANA SINDHU(23U41A0579), N.AMRUTHA(23U41A0598)**. In partial fulfillment of the requirements for award of the Degree of **Bachelor of Technology in Computer Science Engineering**, from **Dadi Institute of Engineering & Technology(A), Anakapalle affiliated to JNTUGV**, accredited by NAAC with 'A' grade is a record of bonafide work carried out by them under my guidance and supervision.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project entitled “**SMART VEHICLE LIGHT**” is submitted in partial fulfilment of the requirements for the award of Bachelor of Technology in **Computer Science & Engineering** under esteemed supervision of **DR.K. SUJATHA , HEAD OF THE DEPARTMENT**. This is a record of work carried out by us and results embodied in this project report have not been submitted to any other university for the award of any Degree.

PROJECT ASSOCIATES

K.JAHNAVI(23U41A0583)

K.CHANDINI(23U41A0588)

P.BHAVANA(23U41A0599)

G.DHANUSHKA(23U41A0578)

G.SINDHU(23U41A05879)

ACKNOWLEDGEMENTS

We would like to express our gratitude to our project guide, **DR.K. SUJATHA, Head Of the Department**, of the **Computer Science & Engineering**, for her guidance and assistance throughout the development of this project. Without her supervision, support, and encouragement, we would not have gained awareness of many new things during our project.

We would like to thank **Principal Dr. R. Vaikunta Rao**, Dadi Institute of Engineering & Technology, for providing the necessary facilities to carry out our project work successfully.

We would like to thank **Dean of Academics, A. Krishna Nag**, Dadi Institute of Engineering & Technology, for providing essential facilities for our project.

We express our gratitude to the Teaching and Non-teaching Staff of the Department of **Computer Science & Engineering**, who have been directly and indirectly involved in this journey, for their encouragement in completing our project.

We would like express our deep sense of gratitude to **Honorable Chairman, Sri Dadi Ratnakar**, of Dadi Institute of Engineering & Technology, for providing necessary facilities to carry out our project work successfully.

Endeavors over a long period can also be successful through constant effort and encouragement. We wish to take this opportunity to express our deep gratitude to all the people who have extended their cooperation in various ways during our project. It is our pleasure to acknowledge the help of all those respected individuals.

PROJECT ASSOCIATES

K.JAHNAVI(23U41A0583)

K.CHANDINI(23U41A0588)

P.BHAVANA(23U41A0599)

G.DHANUSHKA(23U41A0578)

G.SINDHU(23U41A05879)

N.AMRUTHA(23U41A0598)

ABSTRACT

The Smart Vehicle Light represents a significant advancement in cycling technology, designed to enhance the overall safety and visibility of biker riders, especially during low-light conditions. By incorporating sensors and adaptive features, the smart light adjusts its brightness and patterns based on real-time factors such as ambient light, weather, and the bike speed. For example, the light automatically dims when Bike riders in well-lit areas and brightens when approaching dark roads or intersections. This dynamic lighting feature ensures the riders remains visible to others without overburdening battery life. Furthermore, some models of smart bike lights integrate turn signal indicators, providing greater communication with drivers and pedestrians, reducing the likelihood of accidents.

In addition to its safety features, the Smart Bike Light is equipped with Bluetooth connectivity, allowing it to pair with mobile applications. This functionality enables riders to control light settings, monitor battery life, and customize light patterns directly from their smartphones. The app can also track ride data, including distance, time, and speed, while sending real-time alerts for any maintenance needs or safety concerns. For example, if the light detects an abnormal condition, like a malfunction or battery issue, it sends an alert to the bike rider, ensuring timely intervention. This integration provides cyclists with an enhanced experience and additional peace of mind, knowing their equipment is always in optimal condition.

The Smart vehicle Light not only prioritizes functionality but also sustainability. Most smart lights are designed with energy-efficient LED technology and rechargeable batteries, offering long-lasting performance and minimizing environmental impact. The rechargeable feature reduces the need for disposable batteries, making it more eco-friendly option for bike riders. Moreover, the design of the light is typically compact, weather-resistant, and easy to install, ensuring compatibility with various bike types. As urban riding continues to grow in popularity, the Smart vehicle Light offers an essential tool to enhance rider safety, improve visibility, and promote smarter, more sustainable cycling habits for all riders.

KEY WORDS : DYNAMIC LIGHTING , SUSTAINABILITY, COMPATIBILITY

CONTENTS

S.NO	TOPIC NAME
	DECLARATION
	ACKNOWLEDGEMENT
	ABSTRACT
	CONTENT
1.	INTRODUCTION
2.	REQUIREMENT SPECIFICATIONS 2.1 HARDWARE REQUIREMENTS 2.2 SOFTWARE REQUIREMENTS
3.	SYSTEM DESIGN • 3.1 COMPONENT REQUIREMENTS • 3.2 BLOCK DIAGRAM • 3.3 DATA FLOW
4.	WORKING PRINCIPLE
5.	CODE IMPLEMENTATION
6.	TESTING
7.	CONCLUSION
8.	REFERENCES

LIST OF CONTENTS

DECLARATION

ACKNOWLEDGEMENT

ABSTRACT

CONTENT

1* INTRODUCTION

2* REQUIREMENT SPECIFICATIONS

* 2.1 HARDWARE REQUIREMENTS

* 2.2 SOFTWARE REQUIREMENTS

3* SYSTEM DESIGN

* 3.1 COMPONENT REQUIREMENTS

* 3.2 BLOCK DIAGRAM

* 3.3 DATA FLOW

4* WORKING PRINCIPLE

5* CODE IMPLEMENTATION

6* TESTING

7* ESTIMATED COST

8* ADVANTAGES

9* CONCLUSION

10*REFERENCES

INTRODUCTION

Bike riding has become an increasingly popular mode of transportation and recreation, with more people embracing it for commuting, exercise, and leisure.

However, riding bike in low-light or nighttime conditions poses significant safety risks due to limited visibility. Traditional bike lights, while helpful, often lack advanced features that respond to changing environments and rider

behavior. This is where the Smart Bike Light comes into play. By integrating modern technology such as sensors, Bluetooth connectivity, and adaptive

lighting systems, these smart lights offer a higher level of safety, convenience, and performance, addressing many of the limitations of conventional bike lighting.

At the core of the Smart Bike Light is its ability to automatically adjust to various cycling conditions. Using ambient light sensors, motion detectors, and

GPS, the light system adapts to the rider's speed, environment, and time of day.

For instance, it automatically brightens when entering a darker area and dims when moving through well-lit streets, thereby conserving energy and optimizing battery life. Some models also feature turn signals, brake lights, and automatic response to changes in weather conditions, ensuring the cyclist is always visible and prepared for any situation on the road.

Moreover, the Smart Bike Light leverages mobile app connectivity to further enhance the user experience. Bike riders can control the light's settings, monitor battery levels, track rides, and receive alerts for any issues, all through their

smartphones. This integration of technology not only improves safety but also offers cyclists a level of convenience and customization previously unavailable with traditional bike lights. As urban riding continues to rise, the need for smarter, safer, and more efficient riding solutions becomes ever more critical, and the Smart Bike Light is paving the way for a new era of riding technology.

REQUIREMNT SPECIFICATIONS

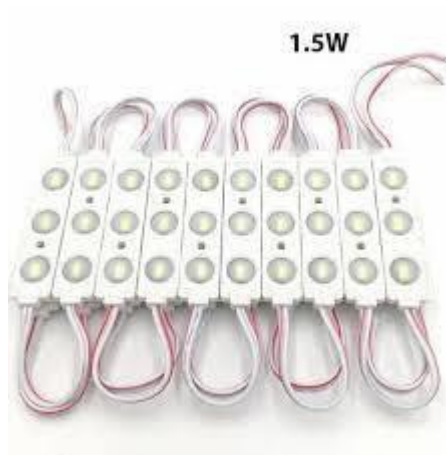
I.HARDWARE REQUIREMENTS:

□ **Microcontroller: Arduino Uno / ESP8266**



NodeMCU is an open-source IoT (Internet of Things) platform based on the ESP8266 or ESP32 microcontroller. It includes both hardware (the development board) and software (a firmware that runs on the ESP chipset

□ LED Light Module



LED light modules are used to create smart lighting systems that can communicate with networks and perform advanced functions. LED lights can be controlled remotely. They can respond to environmental conditions. LED modules are self-contained units that contain multiple light-emitting diodes.

❖ LDR SENSOR



LDRs are used in streetlights, hallway lights, and other outdoor lighting to detect light levels and adjust the lighting accordingly. This saves energy and improves safety. LDRs are used in motion detectors and intrusion alarms to trigger an alarm when a light beam is interrupted.

❖ RESISTORS (10k, 220 ohms)



Resistors are passive electrical components that resist the flow of electrical current in a circuit. They have a variety of uses, including reducing current flow, modifying signal levels, dividing voltages, biasing active devices, and terminating transmission lines.

❖ BREAD BOARD



A breadboard (sometimes called a plug block) is used for building temporary circuits. It is useful to designers because it allows components to be removed and replaced easily. It is useful to the person who wants to build a circuit to demonstrate its action, then to reuse the components in another circuit.

□ JUMPER WIRES



Jumper wires are commonly used with breadboards and other prototyping tools to connect components and test designs. They allow for quick and easy circuit changes without soldering.

□ POWER SUPPLY (USB or Battery)



USB is a standard way to charge batteries in portable devices like MP3 players and PDAs. USB ports are found in many places, including airports, train stations, and public libraries. Charging a device's battery while it's connected to a USB port allows for data exchange and eliminates the need for a wall adapter

II.SOFTWARE REQUIREMENTS

□ ARDUINO IDE



The Arduino IDE (Integrated Development Environment) is a software application used to write, compile, and upload code to Arduino boards and other compatible microcontrollers.

Arduino is widely used in IoT applications for collecting sensor data, processing it, and transmitting it to the cloud for analysis and control, enabling the development of smart and connected devices and systems.

System Design:

The **IoT-based Smart Bike Light** system combines traditional hardware components and modern Internet of Things (IoT) technologies to enhance the functionality, safety, and usability of a bike's lighting system. The primary goal of the IoT-based Smart Bike Light is to ensure safety for cyclists by automating light control, providing visibility in low-light conditions, and enabling remote monitoring or control via IoT connectivity. It leverages sensors, actuators, and connectivity modules to provide an intelligent and interactive lighting solution.

Circuit Components:

Arduino IDE: Used to program the microcontroller (e.g., Arduino Uno, Nano, or ESP8266/ESP32).

LED Light Module: Serves as the bike light.

LDR Sensor: Measures ambient light intensity.

Resistors: Used for the LDR voltage divider circuit and LED current control.

Breadboard: To connect components without soldering.

Jumper Wires: For making connections on the breadboard.

Power Supply: Can be a USB cable, battery, or DC adapter.

Connecting Wires: To bridge components on the breadboard.

Circuit Connection:

Step-by-Step Connections:

Power the Circuit:

- Connect the 5V pin of the Arduino to the power rail on the breadboard.
- Connect the GND pin of the Arduino to the ground rail on the breadboard.

LDR Sensor Circuit:

- Connect one leg of the LDR to the 5V power rail.
- Connect the other leg of the LDR to one side of a resistor (e.g., $10k\Omega$).
- Connect the other side of the resistor to GND.
- Tap into the middle junction (between the LDR and resistor) and connect it to an analog input pin on the Arduino (e.g., A0).

LED Light Module:

- Connect the positive terminal of the LED module to a digital output pin on the Arduino (e.g., D3).
- Place a current-limiting resistor (e.g., 220Ω) in series with the LED to avoid damage.
- Connect the negative terminal of the LED to GND.

Control Logic:

- The Arduino will read the ambient light level from the LDR and control the LED brightness or on/off state based on a threshold.

Jumper Wires:

- Use jumper wires to make all necessary connections on the breadboard.
- Ensure proper insulation and firm connections to avoid loose contacts.

Power the Arduino:

- Use a USB cable connected to a computer or a battery pack.

Block Diagram:

[illegible]

Power Supply

Ambient Light Detection

Arduino & Microntrol

LDR
Sensor

Sensor

Loht
Light
Light Light

LED
Light

LED Light

Limit Definition

Wi-Fi

Bluetooth
Connetation

 Bluetooth

Battery

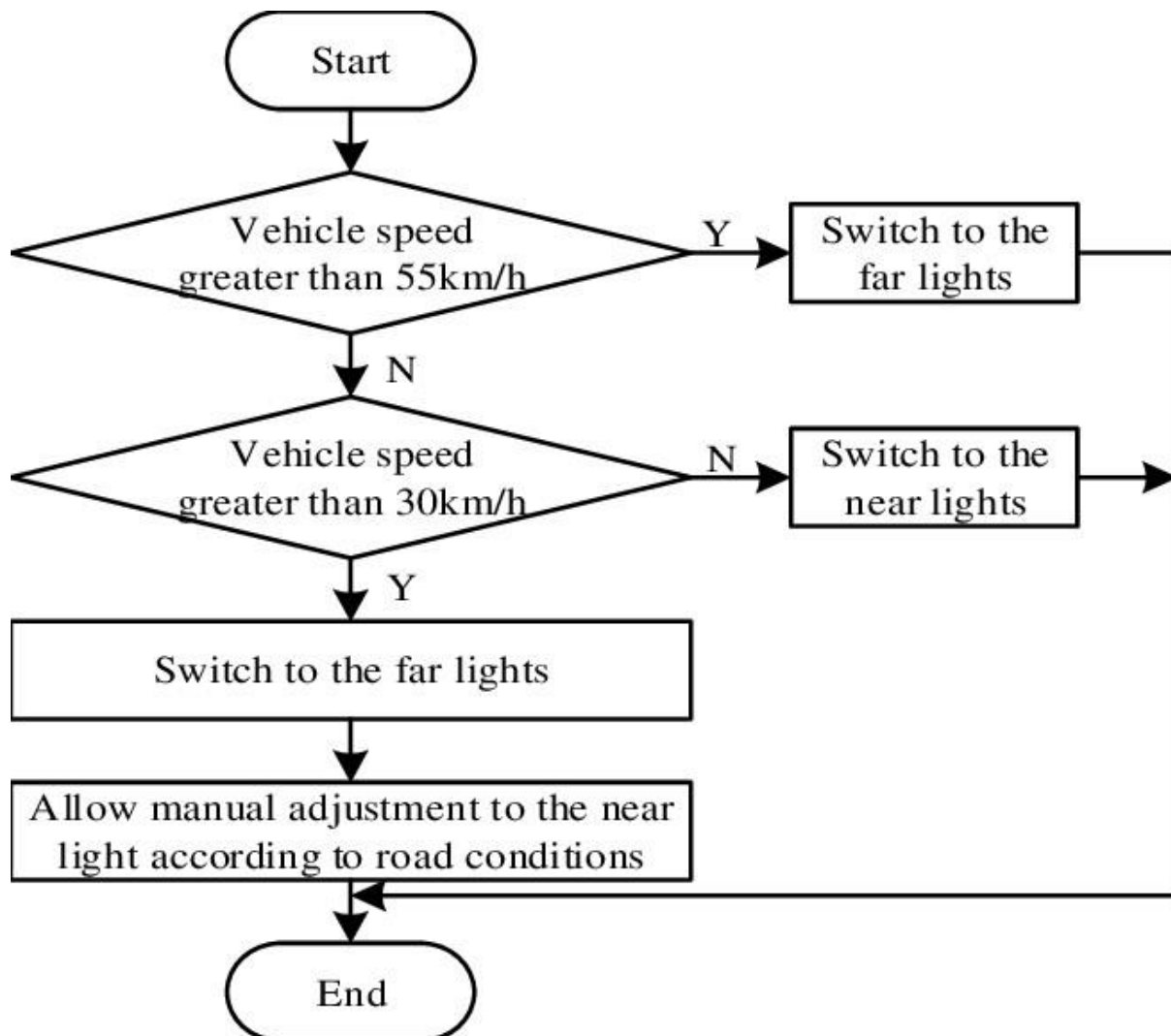
Arduino or
or Epcontroller

LED
Light

Bluecorolla



Bluetooth
Connetation



DATA FLOW:

Sensor Data Collection (Input Stage)

- Speed Sensor: Measures the speed of the bike.
- Accelerometer/Gyroscope: Detects changes in direction, tilt, and motion.
- Light Sensor: Monitors ambient light to adjust brightness.
- GPS Module (Optional): Tracks the location for advanced features (e.g., route tracking, auto light intensity based on time of day).

Microcontroller/Processor (Processing Stage)

- The microcontroller (e.g., Arduino, Raspberry Pi, ESP32) processes the incoming data.

Speed Data: Determines if the bike is moving fast enough for the light to change behaviour (e.g., blink faster when riding faster). oGyroscope/Accelerometer: Detects orientation or turns to adjust light patterns (e.g., blinking when turning). oLight Sensor: Adjusts the brightness of the light based on ambient light conditions. oUser Inputs: Receives manual settings or commands from a mobile app (e.g., to set the light to a specific pattern or turn it off).

Actuator Control (Output Stage)

- Based on the processed data, the smart bike lights adjust:

Brightness: The light intensity varies depending on the ambient light data. oPattern: Blink patterns or solid light based on speed, motion, or direction. oColour: Some lights might change colour for visibility or warning.

Wireless Communication (Optional)

- If your bike light has Bluetooth or Wi-Fi, data can be sent to and received from a mobile app or cloud server.
- Mobile App: Allows the user to manually adjust settings, view data (like speed, distance travelled), or get notifications (e.g., light status, battery level).
- Cloud Server (optional): Stores user data for analysis, stores preferences, or allows remote updates.

Power Management

- The bike light system typically includes a battery (e.g., Li-ion) for powering the lights and sensors.
- The microcontroller manages power consumption to extend battery life, using techniques like turning off sensors when not needed or adjusting light brightness based on the remaining charge.

WORKING PRINCIPLE

An IoT-based smart bike light system works by integrating various sensors, a microcontroller, wireless communication, and IoT connectivity to create an adaptive and responsive lighting solution for cyclists. The system begins by collecting data from sensors, such as a speed sensor, light sensor, accelerometer, and optionally a GPS module. The speed sensor monitors the bike's velocity, allowing the light to adjust its behaviour (e.g., blinking faster at higher speeds).

The light sensor detects ambient light levels, adjusting the bike light's brightness based on the surrounding lighting conditions. The accelerometer detects the bike's motion and orientation, prompting the light to indicate turns or sudden stops by changing patterns or blinking.

This sensor data is then processed by the microcontroller, which uses predefined algorithms to determine the appropriate light behaviour. For example, the microcontroller may increase the light's brightness in dark conditions or adjust the blink rate based on speed. The microcontroller controls the LED lights through a light control circuit, which modulates brightness, colour, or blinking patterns using techniques like PWM (Pulse Width Modulation) for efficient light management.

The system also incorporates wireless communication, typically via Bluetooth or Wi-Fi, to connect to a mobile app. This allows the user to monitor the light's status, adjust settings, and receive notifications (e.g., low battery alerts).

Additionally, the app can enable real-time data collection, such as tracking speed, location, or battery level. Cloud integration may also be used for data storage and remote updates.

Power management is crucial for ensuring the system remains functional over long periods. The system is powered by a rechargeable battery, and the microcontroller ensures energy-efficient operation by entering low-power modes when the bike is stationary or the light is not needed. The app may also notify the user when the battery is running low.

Overall, the system combines data sensing, real-time processing, and wireless communication to create a smart, energy-efficient, and highly customizable bike light that enhances safety and visibility for riders in varying conditions.

CODE IMPLEMENTATION

```
#include <Wire.h>

#include <Adafruit_Sensor.h>

#include <Adafruit_MPU6050.h>

#include <BluetoothSerial.h>

#define FRONT_LED_PIN 15

#define REAR_LED_PIN 2

#define LIGHT_SENSOR_PIN 34 // ADC pin for LDR

#define SPEED_SENSOR_PIN 4 // Digital pin for speed sensor

#define BLUETOOTH_NAME "SmartBikeLight"

// Create objects for sensors

Adafruit_MPU6050 mpu;

BluetoothSerial SerialBT;

float ambientLightLevel = 0; int

bikeSpeed = 0; int prevSpeed = 0;

unsigned long lastSpeedCheck = 0;

void setup() {

  // Initialize Serial Monitor for debugging

  Serial.begin(115200);

  // Initialize LED pins
```

```

pinMode(FRONT_LED_PIN, OUTPUT); pinMode(REAR_LED_PIN,
OUTPUT);

// Initialize Bluetooth
SerialBT.begin(BLUETOOTH_NAME);
Serial.println("Bluetooth Initialized.");

// Initialize Light Sensor (LDR)
pinMode(LIGHT_SENSOR_PIN, INPUT);

// Initialize Speed Sensor (Reed Switch)
pinMode(SPEED_SENSOR_PIN, INPUT_PULLUP);

// Initialize MPU6050 (Accelerometer/Gyroscope)
Wire.begin(); if (!mpu.begin()) {
    Serial.println("Could not find a valid MPU6050 sensor, check wiring.");
    while (1);
}
Serial.println("MPU6050 Initialized.");

// Set LED light to an initial mode digitalWrite(FRONT_LED_PIN,
LOW); // Turn off front light initially digitalWrite(REAR_LED_PIN,
LOW); // Turn off rear light initially
}

void loop() {

```

```
// Read ambient light level using LDR (Light Sensor)  ambientLightLevel =  
analogRead(LIGHT_SENSOR_PIN);
```

```
// Read speed sensor data (calculate bike speed)  bikeSpeed =  
readSpeed();
```

```
// Read accelerometer data (motion detection)  
readMotion();
```

```
// Adjust light intensity based on ambient light  
adjustLightIntensity();
```

```
// Send data to Bluetooth app (real-time data)  sendBluetoothData();
```

```
delay(100); // Wait a little before the next loop iteration  
}
```

```
int readSpeed() {      static unsigned long
```

```
lastMillis = 0;      static unsigned long
```

```
pulseCount = 0;
```

```
    static unsigned long lastPulseTime = 0;
```

```
    if (digitalRead(SPEED_SENSOR_PIN) == LOW) { // If magnet is near the  
                                                sensor
```

```
        pulseCount++;
```

```
lastPulseTime = millis();
```

```
    }
```

```

if (millis() - lastMillis >= 1000) { // Every second, calculate speed  lastMillis =
millis();

    // Assuming 1 pulse = 1 rotation and wheel circumference is known (e.g., 2.1
    meters)

    float speed = (pulseCount * 2.1) / (millis() - lastPulseTime) * 1000; // meters per
    second

    pulseCount = 0; // Reset pulse count
return int(speed * 3.6); // Convert to km/h
} return 0;
}

void readMotion() {  sensors_event_t a,
g, temp;

    mpu.getEvent(&a, &g, &temp);

    // Detect sudden changes or motion to control emergency blinking or turning  if
(abs(a.acceleration.x) > 9.0 || abs(a.acceleration.y) > 9.0) {

        // Example: Sudden movement (could be a fall or sudden stop)

        digitalWrite(REAR_LED_PIN, HIGH); // Emergency blink

        delay(500);

        digitalWrite(REAR_LED_PIN, LOW);

    }

}

void adjustLightIntensity() {

```



```

// Adjust front light brightness based on ambient light level (simple threshold)  if
    (ambientLightLevel < 500) { // Low light, turn on full brightness
        analogWrite(FRONT_LED_PIN, 255); // Max brightness

        } else { // Bright light, dim the front light
            analogWrite(FRONT_LED_PIN, 50); // Dimmed light
        }

// Adjust rear light based on bike speed  if (bikeSpeed > 20) { //
If    speed    >    20    km/h,    blink    rear    light
digitalWrite(REAR_LED_PIN, HIGH);
    delay(200);
    digitalWrite(REAR_LED_PIN, LOW);
    delay(200);
} else {
    Digital Write (REAR_LED_PIN, HIGH); // Steady light for low speed
}
}

void send Bluetooth Data () {
    // Send speed, light level, and battery status to Bluetooth    String data =
    "Speed: " + String (bike Speed) + " km/h\n";    data += "Ambient Light Level:
    " + String (ambient Light Level) + "\n";
    // Send data to Bluetooth Serial
    Serial BT. Print ln(data);
}

```

TESTING:

Hardware Testing

- **Power Supply Test:**

- Ensure that the battery is correctly connected and that the system is powering up properly.
- Measure the voltage at the input pins of the microcontroller and sensors to ensure they are receiving the correct power.

- **LED Testing:**

- Test the front and rear LEDs by manually controlling them using the microcontroller (turn them on and off).
- Verify that the PWM signals are properly controlling brightness (e.g., adjust brightness via the light sensor or manually in the code).
- For rear light blinking, ensure that the blink pattern is responsive to bike speed.

- **Sensor Testing:**

- **Light Sensor (LDR):**
 - Shine a flashlight or expose the LDR to bright light and verify that the front LED's brightness increases.
 - In darker environments, the light should dim appropriately.

Mobile App Integration Test:

- Ensure that the app displays the real-time data (speed, light level, etc.) correctly.
- Test user interaction with the app (adjusting light intensity, turning off the light, etc.) and check if the commands are sent and received correctly via Bluetooth.
- Confirm that notifications (low battery or error messages) are displayed on the app when necessary.

Battery Life Test:

- Monitor the battery voltage over time to check if the system consumes too much power.
- Use a multi meter to measure current draw and verify that it stays within acceptable limits (ideally <100 mA in idle mode).

Real-World Simulation

Ride Test:

- Test the bike light on an actual bike during a ride. This will help validate how the system performs in real-world conditions, including:
 - How the bike light behaves at different speeds.
 - How well the light sensor adjusts the brightness.
 - How accurately the system responds to motion (acceleration and braking).
 - Whether Bluetooth connectivity remains stable during movement.

Weather Conditions:

- Test the system under different environmental conditions (rain, bright sunlight) to check the robustness of the hardware, especially the sensors and battery.

Debugging :

Serial Monitor Debugging:

- Use the Serial Monitor to print real-time data from sensors (e.g., speed, light level, accelerometer values) to verify that the data being read is correct. ○Log important events, like Bluetooth connections, emergency blinking, and light intensity changes, to help identify problems.

ESIMATED COST

S.NO	COMPONENT	ESTIMATED COST RANGE (₹)
1.	LED's	50-500
2.	Micro controller (Arduino nano)	200-800
3.	Battery (Rechargeable)	300-1500
4.	Charging circuit (USB)	100-300
5.	Sensors (e.g: LDR sensor ,light sensor	100-500
6.	Wireless module (e.g Bluetooth)	300-1000
7.	Housing or mounting materials	100-500
8.	Wiring and connectors	50-200

ADVANTAGES

Smart vehicle lighting systems offer a range of advantages that enhance safety, efficiency, and overall driving experience. Here are some key benefits:

*** Enhanced Visibility:**

* Smart lighting, especially with technologies like adaptive LEDs and laser headlights, provides more precise and focused illumination. This significantly improves visibility, particularly at night and in adverse weather conditions.

* Longer illumination ranges, especially with laser headlights, allow drivers to see further down the road, providing more time to react to potential hazards.

*** Improved Safety:**

* Adaptive headlights adjust their beam patterns based on driving conditions, minimizing glare for oncoming drivers while maximizing illumination for the driver.

* Automatic high beams automatically switch between high and low beams, preventing blinding other drivers.

* These systems, when combined with sensors and cameras, can detect obstacles and pedestrians, further enhancing safety.

*** Increased Energy Efficiency:**

* LED and laser headlights consume significantly less energy than traditional halogen bulbs, reducing the vehicle's energy consumption and contributing to fuel efficiency.

*** Advanced Adaptability:**

* Smart lighting systems can adjust light intensity and direction based on various factors, including vehicle speed, steering angle, and surrounding environment.

- * They can also adapt to different driving modes, such as city driving or highway driving, providing optimal illumination in each scenario.

- * **Design Flexibility:**

- * The compact size of LEDs and laser diodes allows for more creative and flexible headlight designs, enhancing the aesthetic appeal of vehicles.

- * **Automation:**

- * smart lighting systems can automatically turn on and off based on ambient light levels. This removes the human error factor of forgetting to turn on headlights.

- * **Reduction of light pollution:**

- * Because smart lighting systems can focus light more precisely, and reduce unneeded light output, they can help to reduce light pollution.

In essence, smart vehicle lighting represents a significant advancement in automotive technology, contributing to safer, more efficient, and more comfortable driving experiences.

CONCLUSION:

The **IoT-based Smart Bike Light** project improves riders safety by automatically adjusting lighting based on environmental and motion data. It uses sensors to detect ambient light, bike speed, and motion, with Bluetooth integration for real-time control via a mobile app. Features like adaptive lighting, speed-dependent blinking, and emergency signal enhance visibility. The project is energy-efficient and offers potential for future upgrades in smart transportation. It combines IoT technologies to create a safer and more convenient cycling experience.

REFERENCES:

https://www.researchgate.net/publication/331374418_IoT_Enabled_Smart_Bicycle_Safety_System

<https://www.ijraset.com/files/serve.php?FID=21416><https://www.mdpi.com/1424-8220/20/5/1473>

<https://www.electronicsforu.com/electronics-projects/smart-bicyclelight>

<https://www.ijraset.com/files/serve.php?FID=21416>